JSON + Python

Quick Reference — Parsing · Writing · Navigating · Normalizing

1. JSON ↔ Python Type Conversions

json.loads() converts JSON types to Python types automatically:

JSON	Python	Example
object	dict	{"key": "val"}
array	list	[1, 2, 3]
string	str	"hello"
number (int)	int	42
number (real)	float	3.14
true	True	—
false	False	—
null	None	—

2. The 4 Core Functions

> *Memory trick: the 's' suffix = string. No suffix = file. Same as pickle, csv, and most Python serialization libs.*

Function	Direction	Source / Dest
json.loads(s)	JSON string → Python	string (in memory)
json.dumps(obj)	Python → JSON string	string (in memory)
json.load(f)	JSON file → Python	file object
json.dump(obj, f)	Python → JSON file	file object

3. Parsing & Serializing Strings

json.loads() — string → Python

```
import json

json_string = '{"name": "Alice", "age": 30}'
data = json.loads(json_string)   # str -> dict

print(data["name"])   # Alice
print(type(data))     # <class 'dict'>
```

json.dumps() — Python → string

```
data = {"name": "Alice", "scores": [95, 87], "active": True}

# Basic
json.dumps(data)

# Pretty-printed + sorted keys
json.dumps(data, indent=4, sort_keys=True)

# Output:
# {
#     "active": true,
#     "name": "Alice",
#     "scores": [95, 87]
# }
```

> *indent=4 makes output human-readable. sort_keys=True makes it deterministic — good for debugging or diffing.*

4. Reading & Writing Files

json.load() — read from file

```
with open('states.json', 'r') as f:
    data = json.load(f)   # file -> dict/list

# Access nested data
first_state = data['states'][0]['name']   # 'Alabama'
```

json.dump() — write to file

```
new_data = {"Alabama": "AL", "Alaska": "AK"}

with open('new_states.json', 'w') as f:
    json.dump(new_data, f, indent=4)
```

> *Use 'w' to overwrite, 'a' to append. Avoid appending raw JSON — it produces invalid JSON. Always write the whole object at once.*

5. Navigating Nested JSON

Chained key & index access

```
# From json_api_response.py — drill into nested structure:
# data['list']['resources'][0]['resource']['fields']['price']

count     = data['list']['meta']['count']
resources = data['list']['resources']    # list of dicts

# First resource's symbol
symbol = resources[0]['resource']['fields']['symbol']
```

Safe access with .get()

```
# Risky — raises KeyError if key is missing
emails = person['emails']

# Safe — returns None by default
emails = person.get('emails')

# Safe with fallback value
emails = person.get('emails', [])

# Check before using
if person.get('emails') is not None:
    print(person['emails'])
```

> *Use dict[key] when you know the key exists. Use .get(key) when the key is optional or could be null.*

6. Looping & Transforming

Extract into a flat dict

```
# From working_with_json_2.py
# states.json -> {"Alabama": "AL", "Alaska": "AK", ...}

new_states = {}
for state in data['states']:
    new_states[state['name']] = state['abbreviation']

# Cleaner: dict comprehension
new_states = {
    s['name']: s['abbreviation']
    for s in data['states']
}
```

Extract from deeply nested resource lists

```
# From json_api_response.py
rates = {}
for item in data['list']['resources']:
    fields = item['resource']['fields']   # extract once
    rates[fields['name']] = fields['price']

# rates = {"INR/USD": "83.12", "EUR/USD": "1.08", ...}
```

> *Always extract the nested object into a local variable (like 'fields') first — avoids repeating the long access chain and is much more readable.*

Mutate fields in a loop

```
# From working_with_json.py — scrub phone numbers
for person in data['people']:
    person['phone'] = ''   # mutates in place

# Dump the modified object back to a string
cleaned = json.dumps(data, indent=4, sort_keys=True)
print(cleaned)
```

> *Mutating a dict key while iterating a list works fine — you're modifying the dict, not the list.*

7. Pandas — pd.json_normalize()

Flatten a list of dicts into a DataFrame

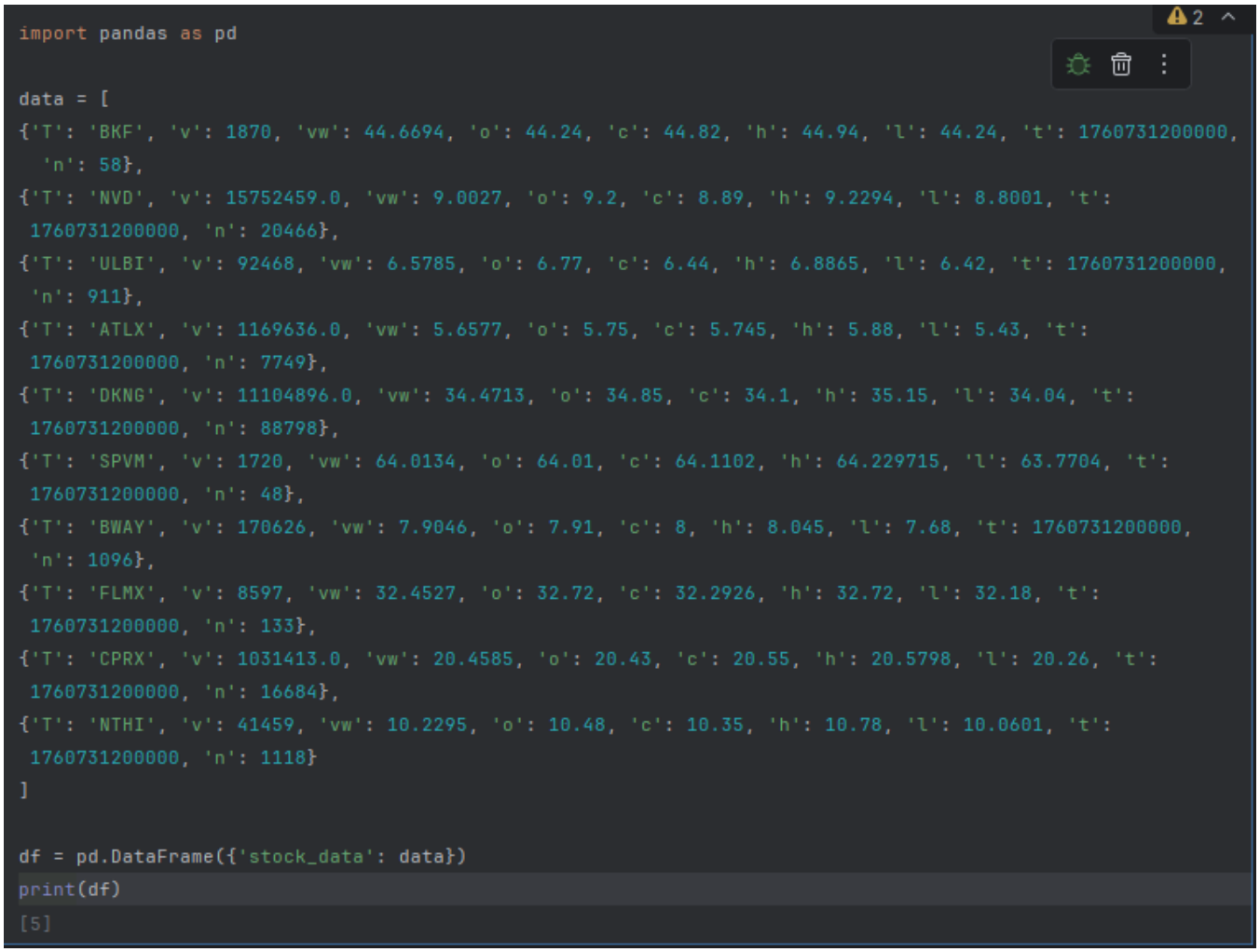
```
import pandas as pd

# From json_normalize.ipynb
# data = list of stock dicts with keys T, v, vw, o, c, h, l, t, n

# Method 1: directly from list of dicts
df = pd.json_normalize(data)

# Method 2: from a column of dicts inside a DataFrame
df = pd.DataFrame({'stock_data': data})
df = pd.json_normalize(df['stock_data'])
```

Eg-



```
df = pd.DataFrame({'stock_data': data})
print(df)
[5]
```

```
                                          stock_data
 0  {'T': 'BKF', 'v': 1870, 'vw': 44.6694, 'o': 44...
 1  {'T': 'NVD', 'v': 15752459.0, 'vw': 9.0027, 'o...
 2  {'T': 'ULBI', 'v': 92468, 'vw': 6.5785, 'o': 6...
 3  {'T': 'ATLX', 'v': 1169636.0, 'vw': 5.6577, 'o...
 4  {'T': 'DKNG', 'v': 11104896.0, 'vw': 34.4713, ...
 5  {'T': 'SPVM', 'v': 1720, 'vw': 64.0134, 'o': 6...
 6  {'T': 'BWAY', 'v': 170626, 'vw': 7.9046, 'o': ...
 7  {'T': 'FLMX', 'v': 8597, 'vw': 32.4527, 'o': 3...
 8  {'T': 'CPRX', 'v': 1031413.0, 'vw': 20.4585, '...
 9  {'T': 'NTHI', 'v': 41459, 'vw': 10.2295, 'o': ...
```

```
df = pd.json_normalize(df['stock_data'])
print(df)
[8]
```

```
      T         v      vw      o      c         h      l  \
 0   BKF    1870.0 44.6694  44.24 44.8200 44.940000 44.2400
 1   NVD 15752459.0  9.0027   9.20  8.8900  9.229400  8.8001
 2  ULBI   92468.0  6.5785   6.77  6.4400  6.886500  6.4200
 3  ATLX 1169636.0  5.6577   5.75  5.7450  5.880000  5.4300
 4  DKNG 11104896.0 34.4713  34.85 34.1000 35.150000 34.0400
 5  SPVM    1720.0 64.0134  64.01 64.1102 64.229715 63.7704
 6  BWAY  170626.0  7.9046   7.91  8.0000  8.045000  7.6800
```

Select and rename columns

```
# Select only the columns you need
df = df[['T', 'o', 'h', 'l', 'c', 'v']]

# Rename to descriptive names
df = df.rename(columns={
    'T': 'symbol',
    'o': 'open',
    'h': 'high',
    'l': 'low',
    'c': 'close',
    'v': 'volume'
})
```

pd.DataFrame() vs json_normalize — when to use which

Method	Use When	Notes
`pd.DataFrame(list)`	Flat list of dicts (no nesting)	Fast, simple

`pd.json_normalize(list)`	`List of dicts with nested objects`	`Flattens one level automatically`
`json_normalize(df[col])`	`Column of dicts inside a DataFrame`	`Common API response pattern`

> *pd.json_normalize() is a superset of pd.DataFrame() for JSON data. Use it by default unless the data is already flat.*

8. Full API Response Pattern

requests → .json() → DataFrame

```
import requests, pandas as pd

try:
    resp = requests.get(url, params=params)
    resp.raise_for_status()       # raises on 4xx/5xx
    data = resp.json()            # parses body -> dict

    # Guard against empty or missing results
    if 'results' not in data or not data['results']:
        print(f'No data for this request')
    else:
        df = pd.json_normalize(data['results'])
        df = df[['T', 'o', 'h', 'l', 'c', 'v']]

except Exception as e:
    print(e)
```

> *resp.json() is equivalent to json.loads(resp.text) — it parses the response body string automatically. Always guard against missing or empty 'results' keys before normalizing.*

9. Quick Tips & Common Gotchas

Pretty-print any JSON object for inspection:

```
print(json.dumps(data, indent=4))
```

Check type after parsing:

```
print(type(data))          # should be dict or list
print(type(data['key']))   # inspect nested types
```

Convert a response string manually (same as resp.json()):

```
data = json.loads(response.text)
```

Verify metadata vs actual count:

```
# From json_api_response.py
print('Metadata count: %d' % data['list']['meta']['count'])
```

```
print('Actual count:   %d' % len(data['list']['resources']))
```

Add computed columns after normalizing:

```
import datetime as dt

df['trade_date'] = start_date
df['ingest_ts']  = dt.datetime.combine(start_date,
dt.datetime.min.time()).timestamp()
df['_src_file']  = output_csv_file
```